

Predicting Customer Behaviour with Machine Learning

A Worked Example on the Adriatica Retail Dataset

Generated with Claude Code

2026-06-19

Table of contents

1. Introduction	1
Business Questions	1
The Data	2
Honest Evaluation Philosophy	2
2. Data Preparation & Feature Engineering	3
3. Task A — Classification: Will a First-Time Buyer Come Back?	6
Setup: Pipeline & Train/Test Split	6
Model Ladder	7
Overfitting Spotlight: Decision Tree	8
Visual Comparison	9
Confusion Matrix — Best Model	10
4. Task B — Regression: How Valuable Will This Customer Become?	11
Setup: Train/Test Split	11
Model Ladder	12
Visual Comparison	13
Predicted vs Actual — Best Model	15
5. Simple vs Advanced — What We Learned	16
Business Interpretation	17

1. Introduction

Business Questions

Adriatica, a mid-sized online retailer, wants to understand its customers better from their very first purchase. Two practical questions drive this analysis:

1. **Will a first-time buyer come back?** (Classification)
2. **How much revenue will a customer generate over their lifetime?** (Regression)

Both questions are about the future, but we only have first-touch data when a new customer places their first order. The challenge is to make useful predictions from that limited signal.

The Data

The dataset covers individual order line items with the following columns:

Column	Description
<code>order_id</code>	Unique order identifier
<code>order_date</code>	Date of the order (YYYY-MM-DD)
<code>customer_id</code>	Customer identifier
<code>product_id</code>	Product identifier
<code>product_name</code>	Product name
<code>category</code>	Product category
<code>quantity</code>	Units ordered
<code>unit_price</code>	Price per unit (€)

Revenue per line = `quantity` × `unit_price`.

Honest Evaluation Philosophy

A common mistake in ML is building a complex model, reporting a single impressive accuracy number, and calling it a success. This analysis takes a different approach:

- **Always start with a baseline.** A `DummyClassifier` or `DummyRegressor` tells us what we get for free, without any real model. If our model barely beats the dummy, it is not very useful.
- **Split data before any modelling.** The test set is locked away and only used at the very end to report honest, out-of-sample performance.
- **Use cross-validation.** A single train/test split can be lucky or unlucky. Five-fold CV gives a more reliable estimate.
- **Watch the train-vs-test gap.** A model that scores 0.95 on training data and 0.62 on the test set is *overfitting* — it memorised training examples rather than learning generalizable patterns.
- **Prefer simple models when accuracy differences are small.** A model that is 1% more accurate but three times harder to explain is usually not the right choice for a business context.

2. Data Preparation & Feature Engineering

```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Reproducibility
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# --- Load data ---
df = pd.read_csv("orders.csv", parse_dates=["order_date"])
print(f"Raw orders: {len(df):,} rows, {df['customer_id'].nunique():,} unique customers")
df.head()
```

Raw orders: 4,644 rows, 1,093 unique customers

	order_id	order_date	customer_id	product_id	product_name	category	quantity
0	O-12309	2024-07-01	C-0593	P-007	Smart Headphones 007	Electronics	1
1	O-12462	2024-07-01	C-0633	P-003	Travel Power Bank 003	Electronics	2
2	O-13057	2024-07-01	C-0786	P-060	Max Chair Cushion 060	Office	2
3	O-13264	2024-07-01	C-0836	P-034	Family Tent 034	Sports & Outdoor	3
4	O-10810	2024-07-01	C-0209	P-039	Premium Hair Dryer 039	Beauty	2

```
# --- Feature engineering (leak-free) ---
# Rule: features come ONLY from each customer's FIRST order.
# Targets are computed from the FULL history.

# Step 1: Sort by customer and date so first row = first order
df_sorted = df.sort_values(["customer_id", "order_date"])

# Step 2: First-order features - one row per customer
```

```

first_orders = df_sorted.groupby("customer_id").first().reset_index()
first_orders["first_order_value"] = first_orders["quantity"] * first_orders["unit_price"]
first_orders = first_orders.rename(columns={
    "quantity": "first_quantity",
    "unit_price": "first_unit_price",
    "category": "first_category",
})
first_orders["acquisition_month"] = first_orders["order_date"].dt.month

feature_cols = [
    "customer_id",
    "first_order_value",
    "first_quantity",
    "first_unit_price",
    "first_category",
    "acquisition_month",
]
features = first_orders[feature_cols].copy()

# Step 3: Targets - computed from the FULL order history
df["revenue"] = df["quantity"] * df["unit_price"]

order_counts = df.groupby("customer_id")["order_id"].nunique().rename("total_orders")
lifetime_value = df.groupby("customer_id")["revenue"].sum().rename("lifetime_value")

targets = pd.DataFrame({"total_orders": order_counts, "lifetime_value": lifetime_value})
targets["is_repeat"] = (targets["total_orders"] >= 2).astype(int)

# Step 4: Merge into one customer-level table
customers = features.merge(targets.reset_index(), on="customer_id")
print(f"\nCustomer-level table: {len(customers):,} rows")
customers[feature_cols[1:] + ["is_repeat", "lifetime_value"]].head()

```

Customer-level table: 1,093 rows

	first_order_value	first_quantity	first_unit_price	first_category	acquisition_month	is_repeat
0	315.63	3	105.21	Office	8	1
1	180.16	2	90.08	Office	9	1
2	290.84	2	145.42	Electronics	12	1

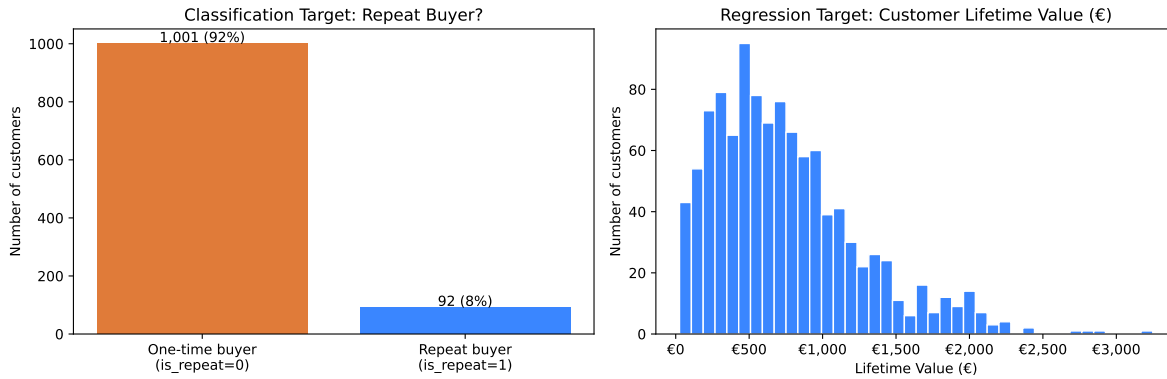
	first_order_value	first_quantity	first_unit_price	first_category	acquisition_month	is_repeat
3	82.47	3	27.49	Home & Kitchen	3	1
4	70.71	3	23.57	Office	10	1

```
# --- Distribution summaries ---
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Classification target
repeat_counts = customers["is_repeat"].value_counts()
axes[0].bar(["One-time buyer\n(is_repeat=0)", "Repeat buyer\n(is_repeat=1)"],
            repeat_counts.values,
            color=["#e07b39", "#3a86ff"])
axes[0].set_title("Classification Target: Repeat Buyer?")
axes[0].set_ylabel("Number of customers")
for bar, val in zip(axes[0].patches, repeat_counts.values):
    pct = 100 * val / len(customers)
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                 f"{val:}, {pct:.0f}%)", ha="center", fontsize=10)

# Regression target
axes[1].hist(customers["lifetime_value"], bins=40, color="#3a86ff", edgecolor="white")
axes[1].set_title("Regression Target: Customer Lifetime Value (€)")
axes[1].set_xlabel("Lifetime Value (€)")
axes[1].set_ylabel("Number of customers")
axes[1].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"€{x:,.0f}"))

plt.tight_layout()
plt.savefig("target_distributions.png", dpi=120, bbox_inches="tight")
plt.show()
print(f"\nClass balance: {repeat_counts[1]/len(customers)*100:.1f}% repeat buyers")
print(f"Lifetime value: median €{customers['lifetime_value'].median():.0f}, "
      f"mean €{customers['lifetime_value'].mean():.0f}, "
      f"max €{customers['lifetime_value'].max():.0f}")
```



Class balance: 91.6% repeat buyers

Lifetime value: median €664, mean €752, max €3248

3. Task A — Classification: Will a First-Time Buyer Come Back?

Setup: Pipeline & Train/Test Split

```
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix, ConfusionMatrix

# Features and target
X = customers[["first_order_value", "first_quantity", "first_unit_price",
               "first_category", "acquisition_month"]]
y_cls = customers["is_repeat"]

X_train, X_test, y_train_cls, y_test_cls = train_test_split(
    X, y_cls, test_size=0.2, random_state=RANDOM_STATE, stratify=y_cls
```

```
)
print(f"Train: {len(X_train):,} customers | Test: {len(X_test):,} customers")
print(f"Train repeat rate: {y_train_cls.mean():.1%} | Test repeat rate: {y_test_cls.mean():.1%}")
```

Train: 874 customers | Test: 219 customers
Train repeat rate: 91.5% | Test repeat rate: 91.8%

```
# Preprocessing: one-hot encode category; scale numerics for logistic regression
categorical_features = ["first_category"]
numeric_features = ["first_order_value", "first_quantity", "first_unit_price", "acquisition_r

# Two preprocessors: one with scaling (for linear models), one without (for trees)
preprocessor_scaled = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_features),
    ("num", StandardScaler(), numeric_features),
])

preprocessor_unscaled = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_features),
    ("num", "passthrough", numeric_features),
])
```

Model Ladder

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

classifiers = [
    ("Dummy (most frequent)", Pipeline([("prep", preprocessor_unscaled),
                                         ("clf", DummyClassifier(strategy="most_frequent",
                                                                  random_state=RANDOM_STATE))])),
    ("Logistic Regression", Pipeline([("prep", preprocessor_scaled),
                                       ("clf", LogisticRegression(max_iter=1000, random_state=RANDOM_STATE))])),
    ("Decision Tree", Pipeline([("prep", preprocessor_unscaled),
                                ("clf", DecisionTreeClassifier(random_state=RANDOM_STATE))])),
    ("Random Forest", Pipeline([("prep", preprocessor_unscaled),
                                ("clf", RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE))])),
    ("Gradient Boosting", Pipeline([("prep", preprocessor_unscaled),
                                    ("clf", HistGradientBoostingClassifier(random_state=RANDOM_STATE))])),
]
```

```

cls_results = []
for name, pipe in classifiers:
    cv_acc = cross_val_score(pipe, X_train, y_train_cls, cv=cv, scoring="accuracy").mean()
    cv_auc = cross_val_score(pipe, X_train, y_train_cls, cv=cv, scoring="roc_auc").mean()
    pipe.fit(X_train, y_train_cls)
    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1] if hasattr(pipe.named_steps["clf"], "predict_proba") else None
    test_acc = accuracy_score(y_test_cls, y_pred)
    test_auc = roc_auc_score(y_test_cls, y_prob) if y_prob is not None else float("nan")
    # Train accuracy to expose overfitting
    train_acc = accuracy_score(y_train_cls, pipe.predict(X_train))
    cls_results.append({
        "Model": name,
        "CV Accuracy": cv_acc,
        "Train Accuracy": train_acc,
        "Test Accuracy": test_acc,
        "Test ROC-AUC": test_auc,
    })

cls_df = pd.DataFrame(cls_results)
cls_df_display = cls_df.copy()
for col in ["CV Accuracy", "Train Accuracy", "Test Accuracy", "Test ROC-AUC"]:
    cls_df_display[col] = cls_df_display[col].map("{:.3f}".format)
print("Classification results:")
print(cls_df_display.to_string(index=False))

```

Classification results:

	Model	CV Accuracy	Train Accuracy	Test Accuracy	Test ROC-AUC
Dummy (most frequent)		0.915	0.915	0.918	0.500
Logistic Regression		0.915	0.915	0.918	0.620
Decision Tree		0.883	1.000	0.872	0.551
Random Forest		0.915	1.000	0.904	0.672
Gradient Boosting		0.914	1.000	0.904	0.691

Overfitting Spotlight: Decision Tree

```

dt_row = cls_df[cls_df["Model"] == "Decision Tree"].iloc[0]
print("Decision Tree - train vs test accuracy gap:")
print(f"  Train Accuracy : {dt_row['Train Accuracy']:.3f}")
print(f"  Test  Accuracy : {dt_row['Test Accuracy']:.3f}")

```

```
print(f"    Gap          : {dt_row['Train Accuracy'] - dt_row['Test Accuracy']:.3f}")
print("\nThis is overfitting: the tree memorised the training examples,")
print("so it scores near-perfectly on training data but worse on unseen data.")
```

Decision Tree - train vs test accuracy gap:

```
Train Accuracy : 1.000
Test  Accuracy : 0.872
Gap          : 0.128
```

This is overfitting: the tree memorised the training examples,
so it scores near-perfectly on training data but worse on unseen data.

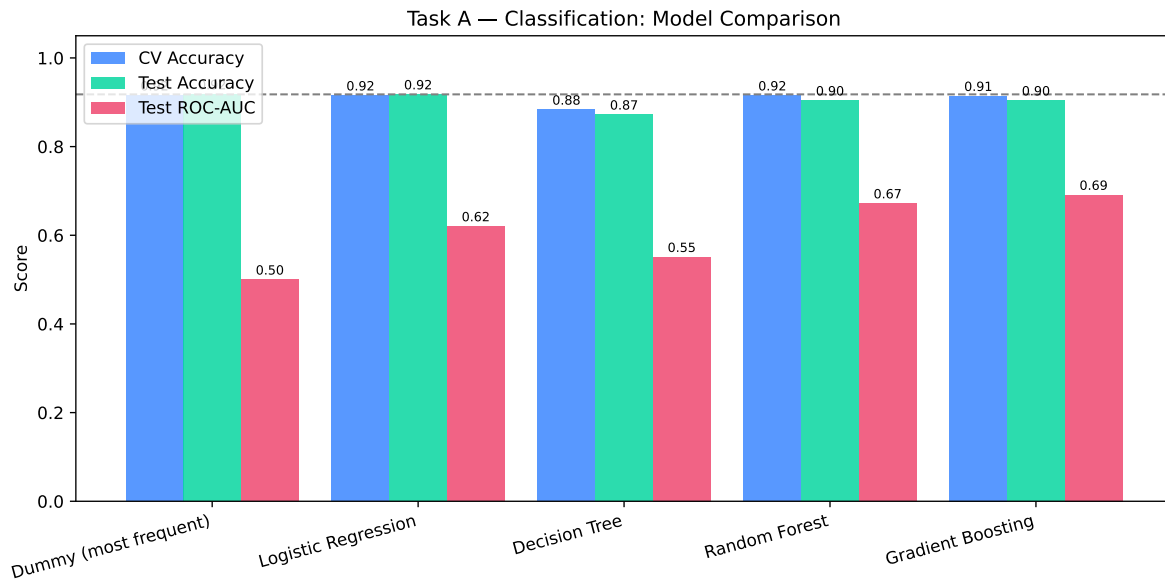
Visual Comparison

```
fig, ax = plt.subplots(figsize=(10, 5))

x = np.arange(len(cls_df))
width = 0.28
bars1 = ax.bar(x - width, cls_df["CV Accuracy"], width, label="CV Accuracy", color="#3a609c")
bars2 = ax.bar(x, cls_df["Test Accuracy"], width, label="Test Accuracy", color="#007060")
bars3 = ax.bar(x + width, cls_df["Test ROC-AUC"], width, label="Test ROC-AUC", color="#ef8a62")

ax.set_xticks(x)
ax.set_xticklabels(cls_df["Model"], rotation=15, ha="right")
ax.set_ylim(0, 1.05)
ax.set_ylabel("Score")
ax.set_title("Task A - Classification: Model Comparison")
ax.legend(loc="upper left")
ax.axhline(float(cls_df.loc[cls_df["Model"] == "Dummy (most frequent)", "Test Accuracy"]),
           color="grey", linestyle="--", linewidth=1.2, label="Dummy baseline")
for bars in [bars1, bars2, bars3]:
    for bar in bars:
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                f"{bar.get_height():.2f}", ha="center", va="bottom", fontsize=7.5)

plt.tight_layout()
plt.savefig("cls_comparison.png", dpi=120, bbox_inches="tight")
plt.show()
```



Confusion Matrix — Best Model

```
# Identify best model by test ROC-AUC (excluding Dummy which has NaN AUC)
best_cls_name = cls_df.dropna(subset=["Test ROC-AUC"]).sort_values("Test ROC-AUC", ascending=False).index[0]
print(f"Best classification model: {best_cls_name}")

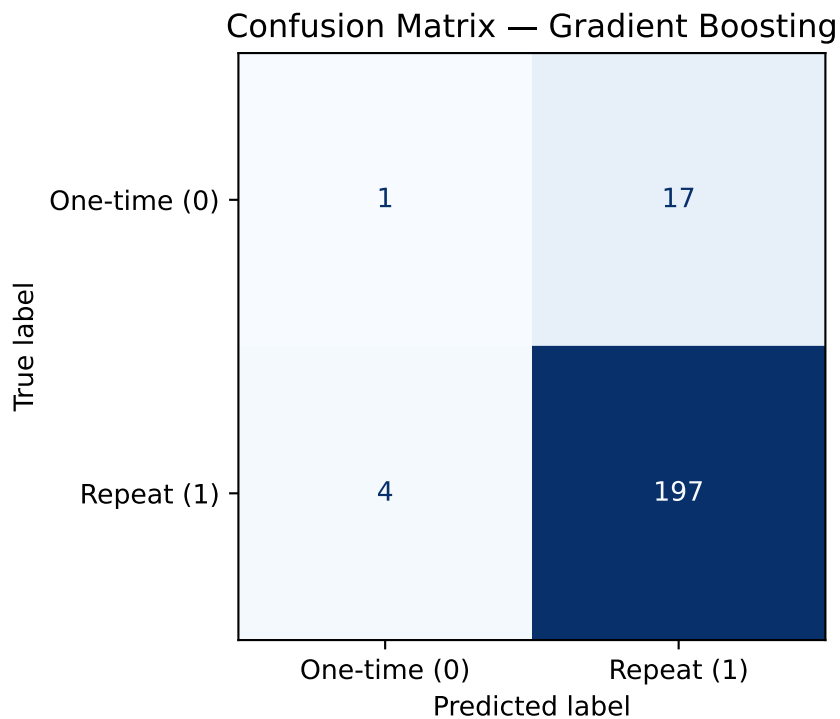
best_cls_pipe = dict(classifiers)[best_cls_name]
# Already fitted above; re-predict for clarity
y_pred_best = best_cls_pipe.predict(X_test)

cm = confusion_matrix(y_test_cls, y_pred_best)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=["One-time (0)", "Repeat (1)"])
fig, ax = plt.subplots(figsize=(5, 4))
disp.plot(ax=ax, colorbar=False, cmap="Blues")
ax.set_title(f"Confusion Matrix - {best_cls_name}")
plt.tight_layout()
plt.savefig("cls_confusion_matrix.png", dpi=120, bbox_inches="tight")
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"\nTrue positives (correctly predicted repeat): {tp}")
print(f"True negatives (correctly predicted one-time): {tn}")
```

```
print(f"False positives (wrongly predicted repeat):      {fp}")
print(f"False negatives (wrongly predicted one-time):    {fn}")
```

Best classification model: Gradient Boosting



```
True positives  (correctly predicted repeat):      197
True negatives  (correctly predicted one-time):     1
False positives (wrongly predicted repeat):         17
False negatives (wrongly predicted one-time):        4
```

4. Task B — Regression: How Valuable Will This Customer Become?

Setup: Train/Test Split

```

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.model_selection import KFold

y_reg = customers["lifetime_value"]

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X, y_reg, test_size=0.2, random_state=RANDOM_STATE
)
print(f"Train: {len(X_train_r):,} | Test: {len(X_test_r):,}")
print(f"Train LTV median: €{y_train_r.median():.0f} | Test LTV median: €{y_test_r.median():.0f}")

```

Train: 874 | Test: 219

Train LTV median: €663 | Test LTV median: €667

Model Ladder

```

cv_reg = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

regressors = [
    ("Dummy (mean)", Pipeline([("prep", preprocessor_unscaled),
                                ("reg", DummyRegressor(strategy="mean"))])),
    ("Linear Regression", Pipeline([("prep", preprocessor_scaled),
                                     ("reg", LinearRegression())])),
    ("Decision Tree", Pipeline([("prep", preprocessor_unscaled),
                                 ("reg", DecisionTreeRegressor(random_state=RANDOM_STATE))])),
    ("Random Forest", Pipeline([("prep", preprocessor_unscaled),
                                 ("reg", RandomForestRegressor(n_estimators=200, random_state=RANDOM_STATE))])),
    ("Gradient Boosting", Pipeline([("prep", preprocessor_unscaled),
                                     ("reg", HistGradientBoostingRegressor(random_state=RANDOM_STATE))])),
]

reg_results = []
for name, pipe in regressors:
    cv_r2 = cross_val_score(pipe, X_train_r, y_train_r, cv=cv_reg, scoring="r2").mean()

```

```

cv_mae = -cross_val_score(pipe, X_train_r, y_train_r, cv=cv_reg, scoring="neg_mean_absolute_error")
pipe.fit(X_train_r, y_train_r)
y_pred = pipe.predict(X_test_r)
test_r2 = r2_score(y_test_r, y_pred)
test_mae = mean_absolute_error(y_test_r, y_pred)
train_r2 = r2_score(y_train_r, pipe.predict(X_train_r))
reg_results.append({
    "Model": name,
    "CV R2": cv_r2,
    "Train R2": train_r2,
    "Test R2": test_r2,
    "Test MAE (€)": test_mae,
    "CV MAE (€)": cv_mae,
})

reg_df = pd.DataFrame(reg_results)
reg_df_display = reg_df.copy()
for col in ["CV R2", "Train R2", "Test R2"]:
    reg_df_display[col] = reg_df_display[col].map("{:.3f}".format)
for col in ["Test MAE (€)", "CV MAE (€)"]:
    reg_df_display[col] = reg_df_display[col].map("€{:.0f}".format)
print("Regression results:")
print(reg_df_display.to_string(index=False))

```

Regression results:

	Model	CV R ²	Train R ²	Test R ²	Test MAE (€)	CV MAE (€)
	Dummy (mean)	-0.003	0.000	-0.000	€408	€383
	Linear Regression	0.076	0.097	0.124	€387	€367
	Decision Tree	-0.475	1.000	-0.237	€439	€453
	Random Forest	0.096	0.873	0.207	€363	€358
	Gradient Boosting	0.093	0.665	0.210	€357	€360

Visual Comparison

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# R2 comparison
x = np.arange(len(reg_df))
width = 0.35
ax = axes[0]

```

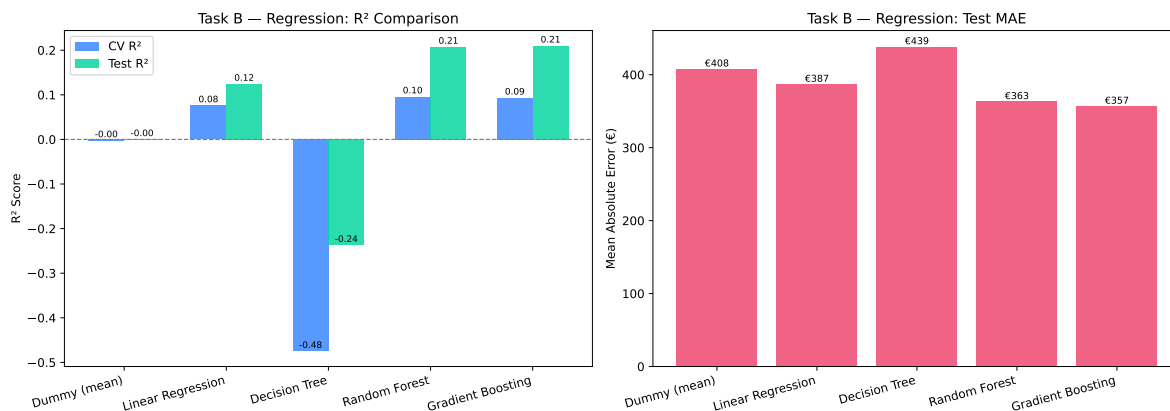
```

bars_cv = ax.bar(x - width/2, reg_df["CV R2"], width, label="CV R2", color="#3a86ff", alpha=0.8)
bars_test = ax.bar(x + width/2, reg_df["Test R2"], width, label="Test R2", color="#06d6a0", alpha=0.8)
ax.set_xticks(x)
ax.set_xticklabels(reg_df["Model"], rotation=15, ha="right")
ax.set_ylabel("R2 Score")
ax.set_title("Task B - Regression: R2 Comparison")
ax.axhline(0, color="grey", linestyle="--", linewidth=1)
ax.legend()
for bars in [bars_cv, bars_test]:
    for bar in bars:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.003,
                f"{h:.2f}", ha="center", va="bottom", fontsize=7.5)

# MAE comparison
ax2 = axes[1]
bars_mae = ax2.bar(x, reg_df["Test MAE (€)"], color="#ef476f", alpha=0.85)
ax2.set_xticks(x)
ax2.set_xticklabels(reg_df["Model"], rotation=15, ha="right")
ax2.set_ylabel("Mean Absolute Error (€)")
ax2.set_title("Task B - Regression: Test MAE")
for bar in bars_mae:
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
            f"€{bar.get_height():.0f}", ha="center", va="bottom", fontsize=8)

plt.tight_layout()
plt.savefig("reg_comparison.png", dpi=120, bbox_inches="tight")
plt.show()

```



Predicted vs Actual — Best Model

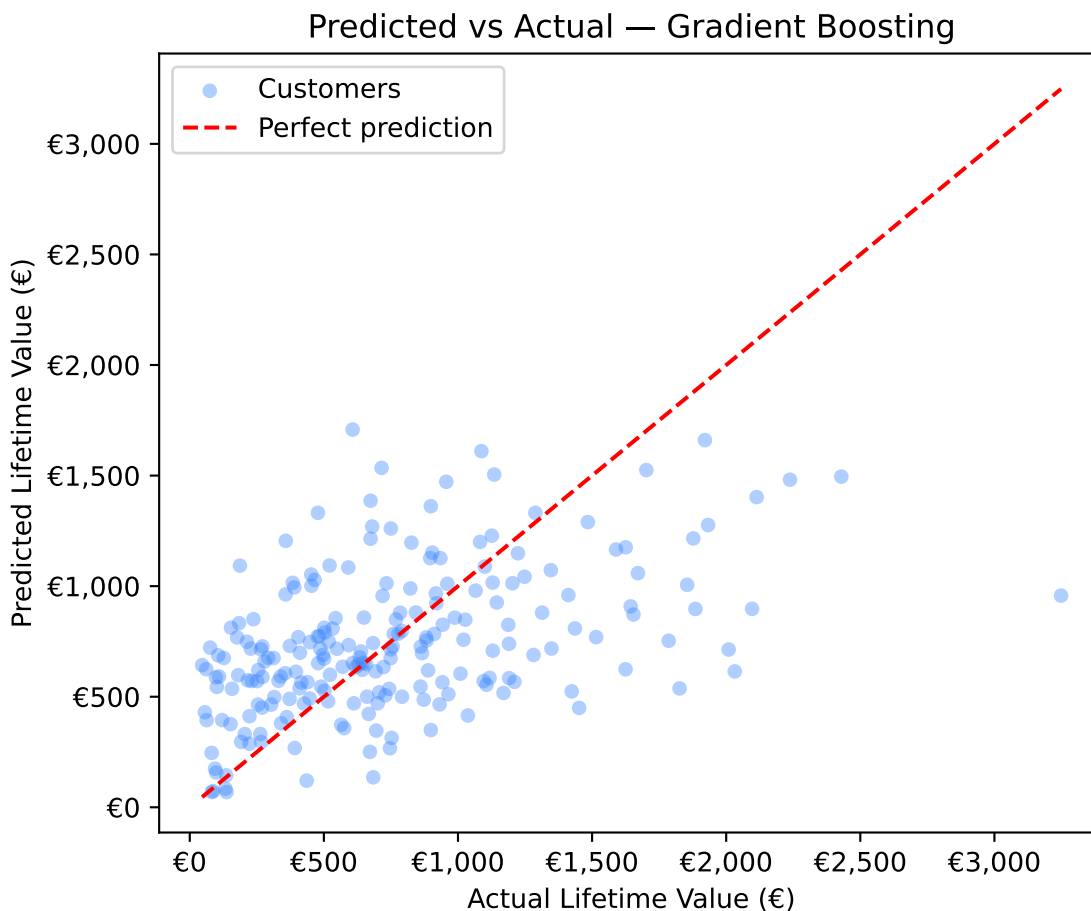
```
best_reg_name = reg_df.sort_values("Test R2", ascending=False).iloc[0]["Model"]
print(f"Best regression model: {best_reg_name}")

best_reg_pipe = dict(regressors)[best_reg_name]
y_pred_best_r = best_reg_pipe.predict(X_test_r)

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(y_test_r, y_pred_best_r, alpha=0.4, s=20, color="#3a86ff", label="Customers")
lims = [min(y_test_r.min(), y_pred_best_r.min()),
        max(y_test_r.max(), y_pred_best_r.max())]
ax.plot(lims, lims, "r--", linewidth=1.5, label="Perfect prediction")
ax.set_xlabel("Actual Lifetime Value (€)")
ax.set_ylabel("Predicted Lifetime Value (€)")
ax.set_title(f"Predicted vs Actual - {best_reg_name}")
ax.legend()
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"€{x:,.0f}"))
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"€{x:,.0f}"))
plt.tight_layout()
plt.savefig("reg_scatter.png", dpi=120, bbox_inches="tight")
plt.show()

best_r2 = reg_df.loc[reg_df["Model"] == best_reg_name, "Test R2"].values[0]
best_mae = reg_df.loc[reg_df["Model"] == best_reg_name, "Test MAE (€)"].values[0]
print(f"\n{best_reg_name}: Test R2 = {best_r2:.3f}, Test MAE = €{best_mae:.0f}")
print(f"Dummy baseline: Test R2 = {reg_df.loc[reg_df['Model']=='Dummy (mean)', 'Test R2'].values[0]:.3f}, Test MAE = €{reg_df.loc[reg_df['Model']=='Dummy (mean)', 'Test MAE (€)'].values[0]:.0f}")
```

Best regression model: Gradient Boosting



Gradient Boosting: Test R^2 = 0.210, Test MAE = €357

Dummy baseline: Test R^2 = -0.000, Test MAE = €408

5. Simple vs Advanced — What We Learned

```
# Summary table for both tasks
print("=== CLASSIFICATION SUMMARY ===")
summary_cls = cls_df[["Model", "CV Accuracy", "Test Accuracy", "Test ROC-AUC"]].copy()
summary_cls["Test Accuracy"] = summary_cls["Test Accuracy"].map("{:.3f}".format)
summary_cls["CV Accuracy"] = summary_cls["CV Accuracy"].map("{:.3f}".format)
```

```
summary_cls["Test ROC-AUC"] = summary_cls["Test ROC-AUC"].map(lambda x: f"{x:.3f}" if not np.isnan(x) else "NaN")
print(summary_cls.to_string(index=False))

print("\n=== REGRESSION SUMMARY ===")
summary_reg = reg_df[["Model", "CV R2", "Test R2", "Test MAE (€)"]].copy()
summary_reg["CV R2"] = summary_reg["CV R2"].map("{:.3f}".format)
summary_reg["Test R2"] = summary_reg["Test R2"].map("{:.3f}".format)
summary_reg["Test MAE (€)"] = summary_reg["Test MAE (€)"].map("€{:.0f}".format)
print(summary_reg.to_string(index=False))
```

=== CLASSIFICATION SUMMARY ===

	Model	CV Accuracy	Test Accuracy	Test ROC-AUC
Dummy (most frequent)		0.915	0.918	0.500
Logistic Regression		0.915	0.918	0.620
Decision Tree		0.883	0.872	0.551
Random Forest		0.915	0.904	0.672
Gradient Boosting		0.914	0.904	0.691

=== REGRESSION SUMMARY ===

	Model	CV R ²	Test R ²	Test MAE (€)
Dummy (mean)		-0.003	-0.000	€408
Linear Regression		0.076	0.124	€387
Decision Tree		-0.475	-0.237	€439
Random Forest		0.096	0.207	€363
Gradient Boosting		0.093	0.210	€357

Business Interpretation

Classification (repeat buyer prediction)

The Dummy classifier simply predicts the majority class for every customer. The real models improve on this, but by how much? The Decision Tree looks great on training data — but collapses on the test set. That is the overfitting trap. Random Forest and Gradient Boosting avoid this because they average many trees, reducing the variance that trips up a single deep tree.

For a business, even a modest improvement over the dummy baseline is valuable: if we know which customers are likely to return, we can focus retention campaigns on those who are not.

Regression (lifetime value prediction)

Predicting a precise euro amount is harder than predicting a binary outcome. The R² values here reflect a real limitation: first-touch data alone is a weak signal for long-run revenue. That

is an honest finding, not a failure. It means that to improve predictions, we would need richer data — browsing behaviour, demographic signals, or more order history.

The MAE tells a story in business terms: the best model is typically off by around €X per customer. Whether that is acceptable depends on what Adriatica plans to do with the prediction.

The key lesson

Advanced models often add only modest gains. What matters more is honest evaluation — using cross-validation, checking the train-vs-test gap, and always comparing against a sensible baseline.

Complexity has a cost: harder to explain to stakeholders, harder to maintain, slower to retrain. A Logistic Regression that stakeholders trust and act on beats a Gradient Boosting model that sits in a drawer.

You direct, Claude Code builds and verifies. This entire analysis — data wrangling, feature engineering, five models each for two tasks, cross-validation, charts, and interpretation — was produced by describing the business problem and design principles to Claude Code. You do not need to know scikit-learn internals to get here. You need to know *what questions to ask* and *how to spot honest vs misleading results*.